

SYSTEM OVERVIEW PDF EDITION

System Overview

Ontology Machine architecture, data path, semantic releases, agents, and workflow boundaries.

Core promise: semantic usefulness without losing the evidence trail.

Prepared from 01_System_Overview(2).md. Revised for readability, clean cards, and rendered flowcharts.

Contents

1. System Overview	3
2. Product Shape	4
3. Core Data Path	5
4. Mainline Modules	6
5. Artifact Tree	8
6. Corpus DB	9
7. Semantic Release	10
8. Base Graph	11
9. Ontology Layer	12
10. Agent Surfaces	13
11. Semantic Control Kernel	14
12. MCP Server	15
13. Client Frontend	16
14. End-To-End Creation Path	17
15. How To Read The System	18
16. Related Chapters	19

Recommended first pass: read sections 1-5, then jump to the agent, Kernel, or creation-path sections that match your work.

1. System Overview

The Ontology Machine is a local Windows-first system for turning document piles into evidence-bound semantic databases. It takes source files, renders and extracts them into durable artifacts, interprets their content with LLM calls, validates the result, normalizes it through an active Semantic Release, and materializes the outcome into a SQLite corpus that can be queried, mined, rebuilt, merged and extended with ontology lenses.

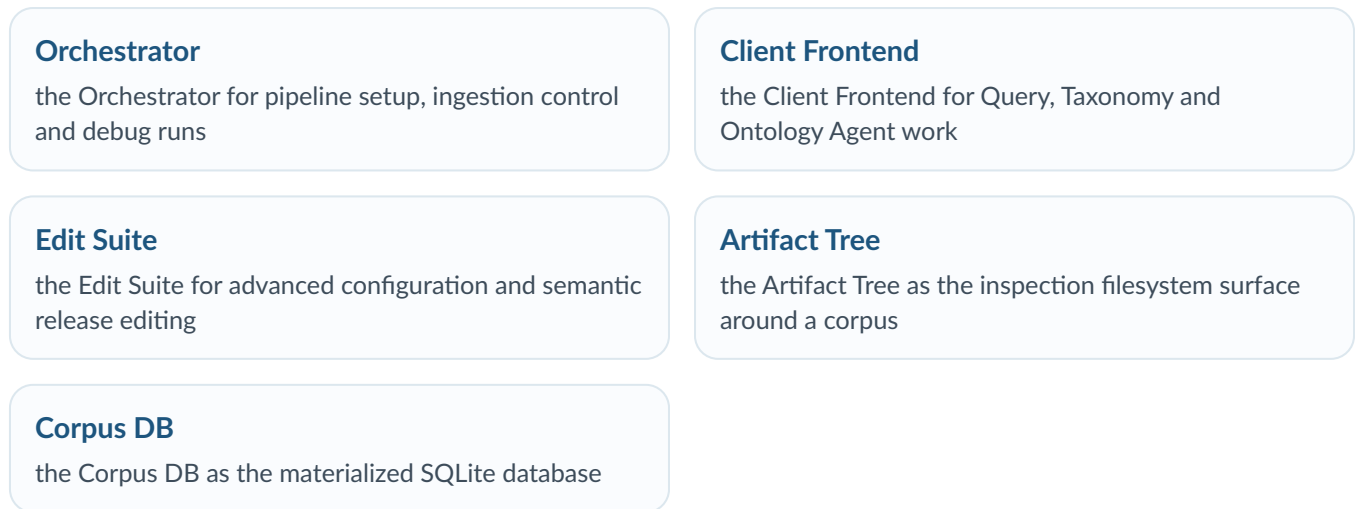
The important thing is that it is not just a chatbot over files. It is a document-to-corpus machine with a visible evidence trail. The chat agents are the user-facing working surfaces on top of that corpus, not the place where the truth of the system secretly lives.



Figure 1. Minimum product model: evidence enters the pipeline, the semantic contract shapes materialization, and agents work on the corpus.

2. Product Shape

The system is shipped as a local all-in-one product. The main user surfaces are:



The codebase is split into modules instead of one large application. The document mainline is:

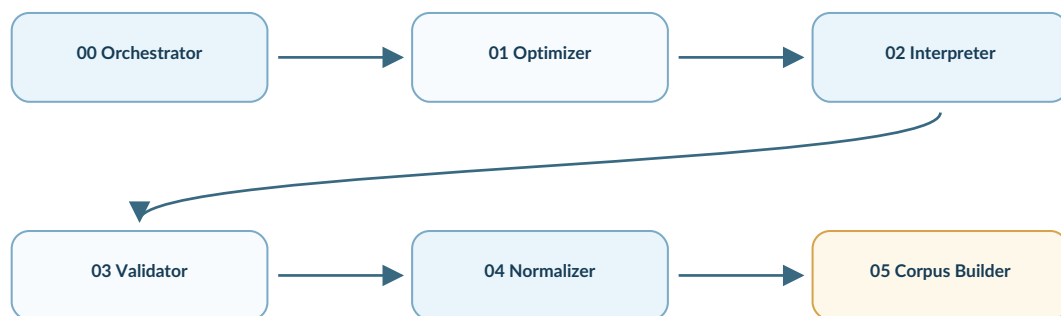


Figure 2. Document ingestion mainline: orchestration through materialization.

The product-level control and interaction surfaces sit around that mainline:

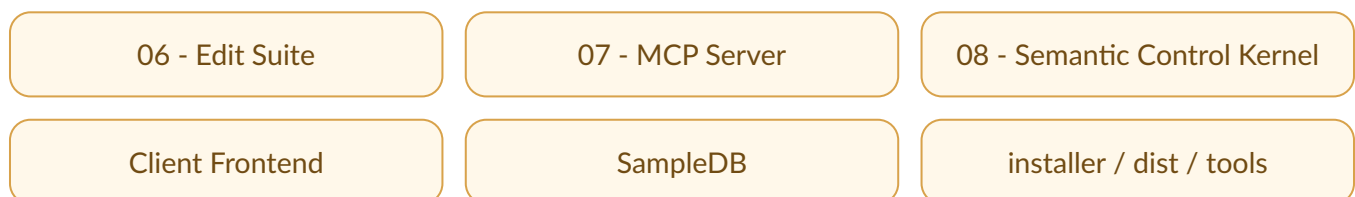


Figure 3. Product-level control and interaction surfaces around the document mainline.

This split matters because the system has several different truth categories: source evidence, generated extraction artifacts, model interpretation, validation output, semantic release contracts, materialized DB rows, runtime workflow state and post-materialized ontology lenses. Keeping those categories separate is what makes the Machine debuggable after a corpus has already been built.

3. Core Data Path

The basic flow is:

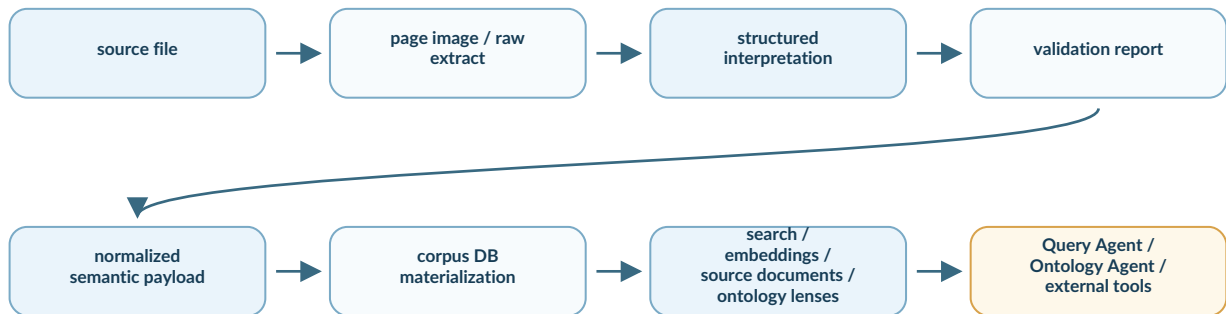


Figure 4. Core data path from source evidence to agent and tool surfaces.

Each step adds a new layer, but the downstream layers do not replace the upstream ones. A normalized row in the database should still be traceable back to structured output, validation, raw extraction and finally the rendered page image or original source.

That is the core promise of the system: semantic usefulness without losing the evidence trail.

Truth categories stay separate

Evidence, extraction artifacts, model interpretation, validation, semantic contracts, DB rows, workflow state, and ontology lenses remain debuggable as different layers.

Agents sit on top

The chat agents expose the corpus to users; they do not become the hidden source of system truth.

4. Mainline Modules

00 - Orchestrator

The Orchestrator is the control plane for the document ingestion mainline. It owns pipeline startup, runtime configuration, model/provider settings for the pipeline modules, artifact tree selection, module debug runs and the operational pipeline state.

It is not the owner of every product feature anymore. Taxonomy creation, long-running creation workflows, ontology mining and normal corpus questioning have their own surfaces now. The Orchestrator remains the mainline operator: when documents move through Optimizer, Interpreter, Validator, Normalizer and Corpus Builder, the Orchestrator is the runtime that coordinates that route.

01 - Optimizer

The Optimizer turns supported source formats into the first machine-readable artifact layer. For born-digital text-like input, this is mostly deterministic local extraction. For image-like input, the V1 product path favors an online LLM OCR/extraction call instead of requiring a local GPU OCR stack.

Its output is intentionally not domain-semantic. The Optimizer should not decide what a document means. Its job is to produce page images, raw extraction payloads and file/page artifacts that downstream modules can use.

02 - Interpreter

The Interpreter answers the question "what do you see?" for the extracted page or document unit. It creates rich structured output from the raw extract and, in vision routes, the page image.

This layer is model interpretation, not final truth. The system allows the Interpreter more expressive freedom than a rigid schema parser, because too much schema pressure at this stage made the output poorer. The following stages are there to validate, normalize and materialize the useful parts.

03 - Validator

The Validator checks whether the structured output still survives contact with the evidence. It reduces silent hallucination, catches numeric/date/value drift where possible, creates validation reports and marks weak cases with review signals.

The Validator is not a perfect truth oracle. Its job is to make failure and uncertainty visible. Hard failures become Error Cases with diagnostic artifacts. Soft uncertainty remains computable through review flags and reasons.

04 - Normalizer

The Normalizer maps structured interpretation into the active Semantic Release. It turns flexible model output into canonical document types, categories, field codes, row types, cell codes, promotion slots and normalized payloads.

The static Normalizer prompt defines general behavior. Domain-specific meaning comes from the active taxonomy and projection. That is why custom releases are so important: a legal archive, an invoice set and a book corpus should not be forced into the same semantic mask.

The Normalizer also owns semantic release management: taxonomy/projection contracts, fingerprints, compatibility rules and activation logic.

05 - Corpus Builder

The Corpus Builder materializes normalized payloads and artifacts into the SQLite Corpus DB. It writes the document rows, payloads, extracted fields, rows, evidence atoms, promotions, entities, page images, embeddings, source-document structure, Base Graph tables and ontology schema.

The Corpus DB is the self-contained query surface. It is not the original source truth, but it carries the links that let a user or agent walk from a claim back to evidence.

5. Artifact Tree

The Artifact Tree is the filesystem surface around a corpus. It is the place where the user or developer can inspect how a database came into existence and where rebuilds can get their input.

The most important top-level folders are:

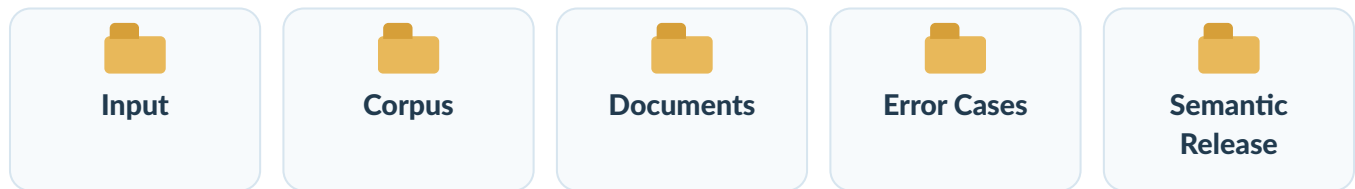


Figure 5. Artifact Tree top-level folders used for ingestion, audit, and rebuilds.

Input is where source files wait before ingestion.

Corpus contains the SQLite database.

Documents contains successful run artifacts such as originals, page images, raw extracts, requests, structured outputs, validation reports, normalized payloads and logs.

Error Cases freezes failed documents or weak cases with enough context to inspect what went wrong.

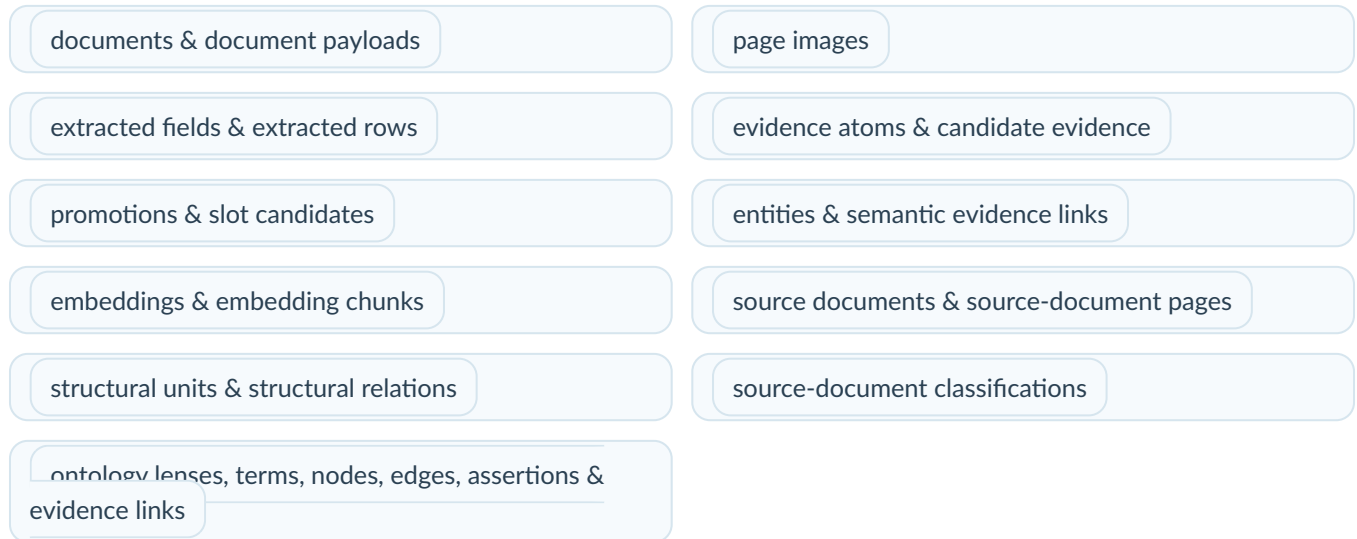
Semantic Release contains the release that shaped the corpus. Without that release, a corpus cannot be interpreted or rebuilt correctly.

The page images inside **Documents\page_images** are the rebuild and audit surface for the Artifact Tree. The Corpus DB also stores page images in **document_page_images** so DB-level evidence links can point directly back to visible page evidence.

6. Corpus DB

The Corpus DB is the materialized semantic state of a corpus. It is designed to be useful on its own: an agent or external tool can query the DB without needing the full pipeline runtime to be active.

Important DB areas include:

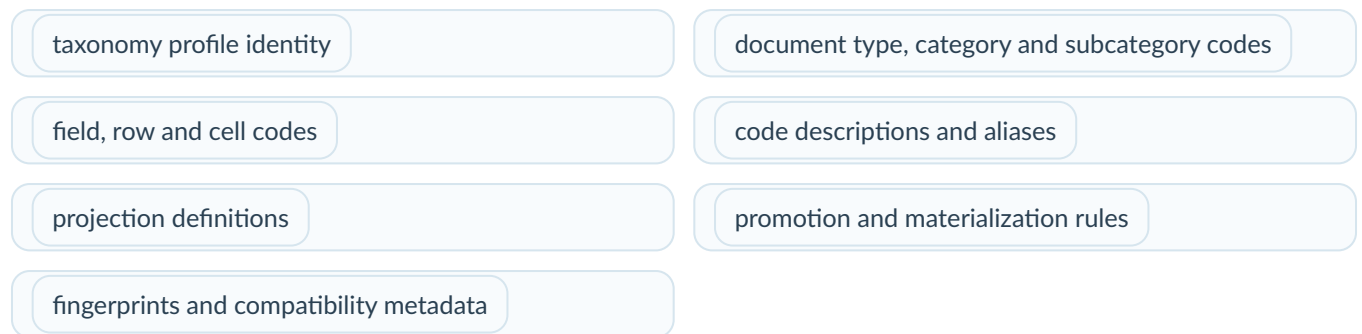


The DB is page-wise at ingestion level, but newer corpora also carry source-document identity fields. The deterministic Base Graph uses those fields to group pages back into source documents and to create the structural layer above page-level materialization.

7. Semantic Release

A Semantic Release is the active semantic contract for a corpus. It defines what the Normalizer is allowed to produce and what the Corpus Builder can materialize.

It contains:



The release is what lets the database behave like a coherent semantic surface instead of a loose pile of LLM-shaped JSON. If two corpora were materialized under incompatible releases, the system must treat that as real drift, not as a cosmetic difference.

8. Base Graph

The Base Graph is the deterministic structure built after page-wise materialization. It does not use an LLM and it does not guess from filenames. It uses source identity fields written by the Corpus Builder, such as `source_document_id`, `source_uri` and `page_index`.

Its purpose is to answer the structural question:



Which page-level DB rows belong to the same source document, and in what order?

Figure 6. The Base Graph answers the structural grouping question after page-wise ingestion.

The Base Graph fills source-document and structural tables such as:

- `source_documents`
- `source_document_pages`
- `structural_units`
- `structural_unit_relations`

This is the layer that makes a multi-page source readable as one document again. It is separate from ontology lenses. Base Graph means deterministic source structure; ontology lenses mean post-materialized interpretation.

9. Ontology Layer

The ontology layer is the Knowledge Mining Layer of the Machine.

It is where the Ontology Agent can build persistent, evidence-bound interpretive graphs over the same corpus. Those graphs are called ontology lenses.

An ontology lens can represent:

a story-arc reading of a novel

a ruthless critique lens over the same novel

a reviewer lens over scientific papers

a legal case theory lens

an audit or correction lens over materialized facts

a worldview or framing lens over news material

any other structured perspective the user wants to preserve

The crucial boundary is that ontology lenses do not overwrite the base corpus. If the base materialized fact says X and a correction lens says the evidence suggests Y, both can remain visible. That is safer and more useful than giving an agent broad write access to the extraction path or canonical truth layer.

10. Agent Surfaces

The Client Frontend exposes three main agent roles.

Query Agent

The Query Agent is read-only. It can inspect the active corpus through SQL, document loading, provenance, semantic search, source-document readers, coverage snapshots, ontology readers and a restricted read-only workbench.

It is designed to answer with evidence. The frontend compares source links in the agent answer against sources actually touched by the tools in the current turn. If the answer mentions a source that cannot be resolved against the current source list, the unresolved source is marked visually. This gives the user a visible warning for possible hallucinated sources or context leakage.

Taxonomy Agent

The Taxonomy Agent is the user-facing surface for Semantic Control Kernel workflows. It is not supposed to manually improvise long tool chains. It selects Kernel workflows, explains dialogs, reports progress and summarizes receipts.

The workflows cover creation routes, custom taxonomy/projection creation, manual ingestion runs, database merge, rebuild, reset, status, resume and cancel operations.

Ontology Agent

The Ontology Agent can read the corpus like the Query Agent and can write only through controlled ontology/base-graph tools.

Its two mutating surfaces are:

- `basic_relation_mining` for deterministic Base Graph construction
- `sql_batch_execute` for preflight-validated ontology-layer writes

`sql_batch_execute` applies write allowlists, required IDs, JSON defaults, parent-first checks, same-lens checks, evidence target checks, edit logging and post-write validation. The Agent can build meaning, but it cannot freely rewrite the whole database.

11. Semantic Control Kernel

The Semantic Control Kernel is the workflow brain behind the Taxonomy Agent. It owns workflow state, dialogs, blockers, progress, resume, recovery and receipts.

The Kernel exists because chat context is the wrong place to store workflow truth. Creating a custom semantic release, merging databases, rebuilding a corpus or running ingestion requires more than a sequence of tool calls. It requires state, confirmations, restart behavior, error classification and a final receipt that says what actually happened.

The Kernel coordinates work but does not replace the owner modules. If a DB must be created or merged, the Corpus Builder still owns the DB primitive. If a release must be compiled or validated, the Normalizer still owns that semantic contract. If provider calls or pipeline runtime state are needed, the Orchestrator remains the runtime surface.

12. MCP Server

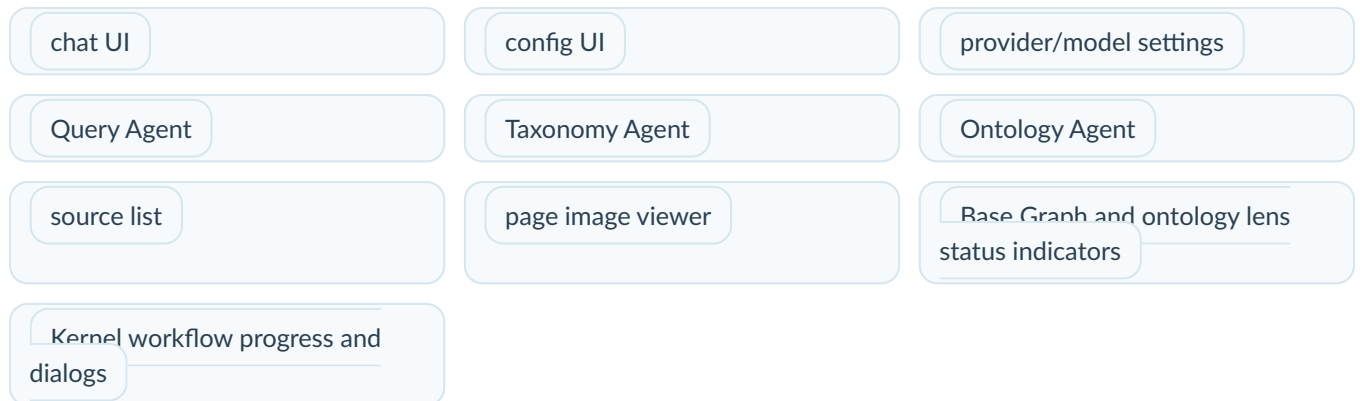
The MCP Server is the local tool bridge. It speaks MCP over standard input/output, exposes a tool catalog and delegates real work to owner-local contracts.

It is not another hidden business-logic host. Its value is that agents can reach the installed system through named tools, schemas and permission-aware routes instead of direct file or database access.

The Semantic Control Kernel is also surfaced through the MCP Server so the Taxonomy Agent can see Kernel workflows as tools while the actual workflow state stays inside the Kernel.

13. Client Frontend

The Client Frontend is the browser-based working surface for normal use. It contains:



The frontend makes the corpus usable without requiring the user to open SQLite, read artifact folders manually or know which Kernel workflow should be called next.

14. End-To-End Creation Path

A common fully custom run looks like this:



Figure 7. Common fully custom creation path from user request to queryable corpus.

The pipeline can also start from default releases, empty DB shells, rebuilds or database merges, but the same product idea remains: create or select a semantic contract, materialize documents under that contract, then query and mine the result.

15. How To Read The System

When taking over the project, it helps to read the Machine by layers instead of by UI buttons:

- 1 Source and page evidence live in the Artifact Tree and DB page-image table.
- 2 Raw extraction belongs to the Optimizer.
- 3 Structured interpretation belongs to the Interpreter.
- 4 Validation reports and review signals belong to the Validator.
- 5 Canonical semantic payloads and releases belong to the Normalizer.
- 6 SQLite materialization belongs to the Corpus Builder.
- 7 Workflow state belongs to the Semantic Control Kernel.
- 8 User interaction and agent sessions belong to the Client Frontend.
- 9 Post-materialized knowledge belongs to ontology lenses.

This is the mental model that makes the system understandable. If a query answer is wrong, the fix is not automatically in the Query Agent. The wrongness may have entered as OCR noise, interpretation drift, validation weakness, normalization loss, materialization shape, missing Base Graph, missing ontology context or source-link mismatch. The correct layer has to be identified before the correct fix becomes obvious.

16. Related Chapters

The Design	Architecture Map
Module Catalog	Contract Library
Workflow Catalog	Artifact Tree Guide
Database Documentation	Agent Documentation
Configuration & Credentials	Production Handover Notes